

Quadratic Assignment Problem – Local Search, Random Methods and Metaheuristics

Aliaksandr Kladneu, Master’s student, PUT

August 28, 2026

1 Introduction

The quadratic assignment problem (QAP) is one of the fundamental combinatorial optimization problems in optimization and operations research.

It can be described as follows: given a set of n facilities and n locations, minimize the sum of distances multiplied by flow between facilities (cost function), by assigning facilities to a dedicated location. Mathematically this can be represented as [1]:

$$\min_{\pi} \leftarrow c(\pi) := \sum a_{ij} b_{\pi(i)\pi(j)}$$

where $A = (a_{ij}) \geq 0$ – matrix of distances between locations and $B_{\pi} = (b_{\pi(i)\pi(j)})$ – is a permutation π of rows and columns of matrix of flows between facilities $B = (b_{ij}) \geq 0$, and $c(\pi)$ is the cost of permutation π , i.e. the value of the objective function for the corresponding assignment.

This is an **NP-hard** problem since the solution space has $n!$ permutations, and even small changes to an assignment affect many terms in the objective value, thus yielding a lot of local optima. Thus it’s usually not feasible to find an optimal solution for large n .

The applications of the problem include circuit design (placing interconnected electronic components onto a printed circuit board) and flow problems (optimization of flows between facilities) [2]. QAP also generalizes TSP and many other standard combinatorial optimization problems.

1.1 QAP Instances

The following instances were chosen, which are available in [3]:

With known optimal solution:

- esc16d
- bur26h
- esc32e
- lipa40a
- lipa80b

With best solution found using heuristics (gap between known lower bound and best cost %):

- tai80b (12.28%)
- wil100 (3.15%), tai100b (10.78%)
- tai150b (11.45%), tho150 (6.3%)
- tai256c (2.03%)

The problems of different sizes were chosen to investigate the impact of problem size n on the quality of solutions and performance of algorithms. Additionally, “twin” problems were added to investigate how a problem’s structure influences these characteristics.

Worth noticing that two of these problems appeared to be “flat”, that is sparse or mostly constant, which means one of two things: it’s very easy to construct almost an ideal solution (minimal cost, *esc32e*) or it’s infeasible or impossible to decrease the cost (*lipa40*). More detailed list of the properties of the selected problems is given in the *Appendix*.

1.1.1 Description of the problems

- *esc-x* problems correspond to the circuit testing problem.
- *tai-b* are uniformly generated random problems, *tai-c* is found in analysis of grey patterns.
- *lipa-x* are generated asymmetric problems with known optimal solutions.
- *bur-x* problem represents the time and distance of typing on a keyboard.
- *tho-x* and *wil-x* problems have rectangular distances.

1.2 Implementation

The implementation is in C++ (20) and is modular: data loading, problem representation, permutation operations, cost/delta evaluation, search algorithms, quality metrics, matrices, statistics, and plotting are separated into dedicated components.

Several methods are implemented and compared: heuristic local search (**H**), steepest local search (**S**), greedy local search (**G**), random search (**RS**), and random walk (**RW**); simulated annealing (with geometric schedule – **SA**, and with Lam schedule – adaptive simulated annealing – **LAMSA**), tabu search (**TS**). For convenience, these methods will be referred to by their provided abbreviations. All these methods, excluding **RS**, reuse information from the current neighbourhood, and thus will be referred to as *local methods* if mentioned together, while **SA**, **LAMSA** and **TS** form a group of *metaheuristics*. The algorithms are implemented in the *algorithms.cpp* module.

Thorough characterization of the heuristic algorithm is given in Section 7, simulated annealing and tabu search are described in Section 8.

Multiple runs of each method were executed in parallel on a machine with 32 logical processors, using *OpenMP* library. Each thread executes the problems in the same order, with random generators independent for each thread. The “wall-clock” timing function provided by *OpenMP* is used to measure the “practical” time of the execution as opposed to measuring cpu time.

Methods in *statistics.cpp* module aggregate data from multiple experiments (runs), while module *plotting.cpp* provides functions for plotting using a *Matplotlib* wrapper [4].

In the second version of the current work, calculating change in cost (‘delta cost’) in the module *cost.cpp* was significantly accelerated by reusing calculated row and column pointers for faster indexing across scattered matrix rows and columns. This change dramatically decreased overall running time.

1.2.1 Experiment Setup

In Section 3, experiments were conducted as follows:

For each problem instance:

1. **H**, **S** and **G** were *rerun 10 times*. Each algorithm stops naturally once search trajectory is exhausted, then restarted internally – in total 10 or 100 times as described in Section 1.2.2.
2. Primary benchmark budget – **mean** time taken by all the 3 methods, over *10 reruns*, was calculated and used as a **time budget** to run **RW**, **RS** and *metaheuristics*.
3. Results of all the methods are aggregated across *10 reruns* and key metrics are analysed.
 - (a) Internally, **H**, **S**, **G** and **RW** aggregate key metrics over multiple *restarts* and return these values each *rerun*.
 - (b) The other methods return one-restart values.
4. Analysis of *efficiency* employs two different time budgets (Section 3.3):
 - **Strict** budget – **median** runtime of **H** (short time window)
 - **Generous** budget – **median** runtime of **S** (extended time window)

In the Sections 4, 5 and 6 1000 independent parallel runs of **G** and **S** were performed for 2 problems – *lipa80b* and *tai100b*, with no internal restarts as in contrast to Section 3.

1.2.2 Multi-Restarts Implementation

In the third version, specifically for the analysis in Section 3, multi-restart loop was added to the *definitions* of each of the following algorithms: **H** (100 times), **S**, **G** and **RW** (10 times each), drawing a new initial permutation by reshuffling the current one and restarting an algorithm again, while updating the best solution so far and its associated cost whenever a restart yields an improvement. The *efficiency* is updated (AUC curve decreases) across all the restarts with each new best cost whenever achieved, and number of swaps and evaluated solutions is cumulative. Each algorithm returns these aggregated across all the restarts values.

Hence, the effective number of repetitions of the algorithms was changed as follows: 1000 times for **H** and 100 times for **S**, **G** and **RW**, though these internal restarts are “hidden”, being opaque to the caller. By applying multiple independent restarts to **G** and **S**, they effectively operate as “*Multi-Start Local Search*” algorithms.

In contrast to **H**, **S** and **G**, **RW** is provided with time budget, which is equally split between restarts: each receives one tenth of the budget. Definition of **RS** was not changed at all since restarting the main loop multiple times is equivalent to the increasing time budget accordingly.

The reader should not confuse *reruns* implemented in *statistics.cpp* module and *restarts* in *algorithms.cpp*. The *former* can be parallelized and are fully independent, while the *latter* are intentionally not parallelized and sequentially update internal statistics (such as the best cost).

The purpose of this modification is to establish a fair, standardized computational budget across all paradigms: pure local search is extremely exploitative and terminates early at a single local optimum, while *metaheuristics* inherently require extended time for exploration. The total execution time of *multi-start local search* algorithms – **H**, **S** and **G** – runs then defines the time budget allocated to a single run of **SA**, **LAMSA**, **TS**, and **RS** (Section 1.2.1). Hence *metaheuristics* and **RS** are now given increased approximately 10 times time budget compared to the *previous* version of this work. This enables a direct, fair comparison between restart based exploration and trajectory-based metaheuristic exploration under identical wall-clock time.

Thus, one of the main questions of the current study – *which of the algorithms is most efficient and worth spending resources on?* – can be answered fairly.

2 Neighbourhood

Neighbourhood operator used in all the *local methods* – pairwise $swap(i, j)$ on a permutation. A neighbor is obtained by selecting two positions i and j and exchanging the assigned locations/facilities at those positions.

The neighbourhood size is the number of possible swaps – number of unique unordered pairs, which is calculated as:

$$|N(x)| = C_2^N = \frac{n(n-1)}{2}$$

Neighbor cost calculation in *local methods* is done using a delta update: instead of recomputing the full QAP objective for each neighbor, one computes the change in objective after swapping i and j using a delta formula [5]. If π is the current solution and π' is the solution after $swap(i, j)$, then

$$c(\pi') = c(\pi) + \Delta_{ij}$$

where Δ_{ij} is computed from terms involving rows/columns i, j and $\forall k \neq i, j$. This reduces one neighbor evaluation from full objective recomputation to an incremental update, making local-search iterations significantly faster (especially for larger n).

3 Comparison of the Performance of the Algorithms on All Problem Instances

3.0.1 Reporting results and statistical considerations

The comparison of algorithms is mainly done using box-plots for different methods and problems. Since for some statistics (*time, norm_cost*) distributions can't have negative values and high deviations in the results are expected, the median value with IQRs are reported as box-plots instead of reporting mean values with confidence intervals.

When providing the plots, the results are sorted by the size of the problem, if not specified otherwise.

3.1 Quality

The first quality metric for comparison of the algorithms is the *relative cost* defined as

$$rel_cost(\pi) := \frac{c(\pi) - c_{best}}{c_{best}} \in [0; +\infty)$$

where π is a solution found by an algorithm, c_{best} is the best cost – cost of the best known solution π_{best} . One can interpret this metric as a (percentage) gap between the cost of found solution and the best known cost: the lower the value the better the quality of the solution regarding to the best known cost. *It is assumed that $c(\pi)$ is not lower than c_{best} .*

The objective function $c(\pi)$ can be thought of as Frobenius scalar product of matrices A and B_π , which following Cauchy-Bunyakovsky-Schwarz inequality, should yield the following definition of the secondary quality metric – *normalized cost*:

$$norm_cost(\pi) := \frac{c(\pi)}{\|A\|_F \|B_\pi\|_F} \equiv \frac{\langle A, B_\pi \rangle_F}{\|A\|_F \|B_\pi\|_F} \equiv \frac{\langle A, B_\pi \rangle_F}{\|A\|_F \|B\|_F} \in [0; 1] \quad (\forall A, B \geq 0)$$

This is referred to as *cos quality* in the code since it's equal to the cosine of the angle between matrices in Frobenius terms. This is useful metric since it does not require the “ground truth” optimal value for comparison and allows direct comparison of different problems, keeping values below 1.

Visual examination of the Figures 1 and 2 reveals significant superiority of **G**, **S**, **SA**, **LAMSA**, and **TS** to solve the problem against proposed simple heuristic **H** and random algorithms **RW**, **RS**. In all of the given problem instances but one (*lipa80b*) the former algorithms achieve best known solution with less than 10% precision (Figure 1), where **G** in many cases demonstrates slightly better quality values *in median* probably due to better exploration properties. At the same time, though unstably – *in several runs out of ten*, **SA** achieves the best cost (*tai80b, tai100b, will100, tai150b, tho150*) in some runs (though median cost is usually higher than one of **G**).

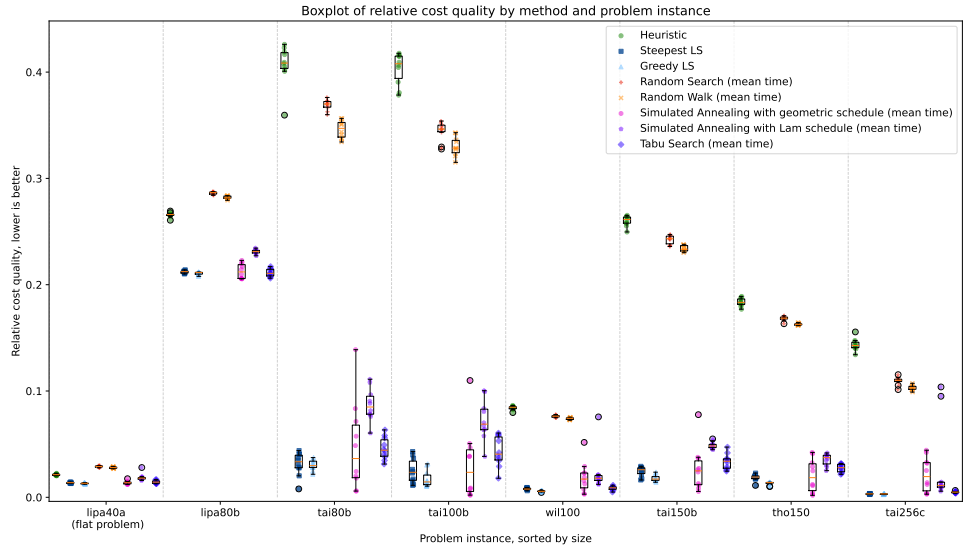


Figure 1: (truncated) Relative cost

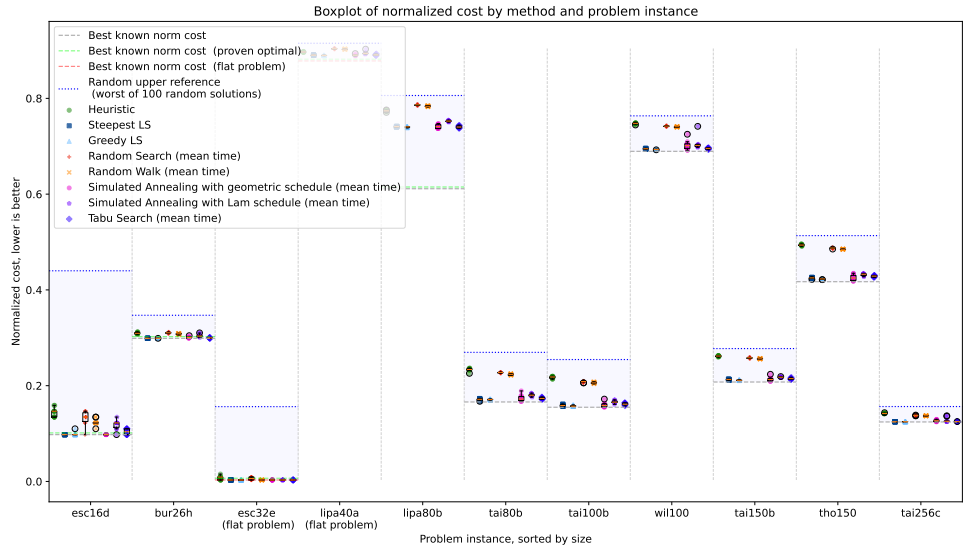


Figure 2: Normalized cost

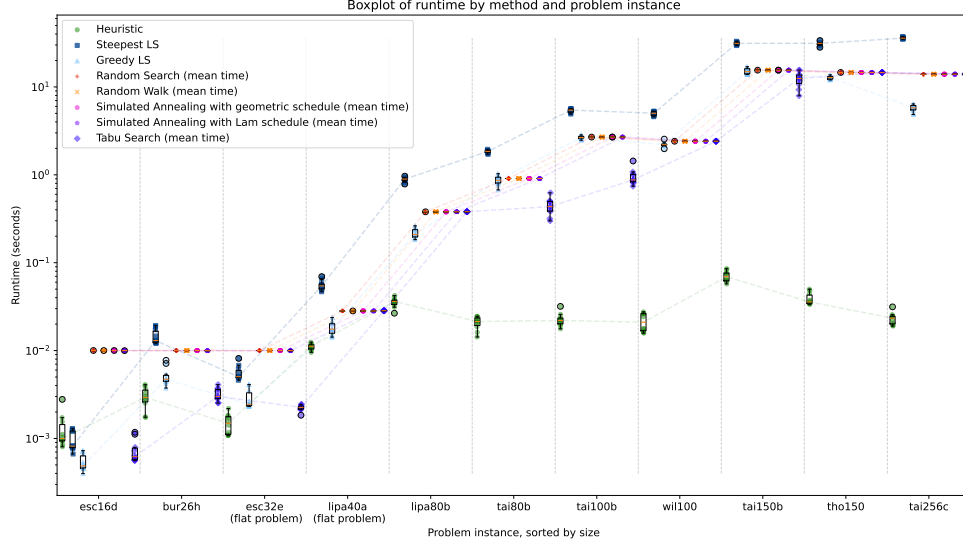


Figure 3: Running time

3.2 Running Time

Figure 3 demonstrates, as expected, that time taken by **H** is the lowest one, while **S** requires the most time to thoroughly examine all the solutions in the neighbourhood, and **G** balances quality and time consumption. Random methods and *metaheuristics* were given mean time of the runs of **H**, **G** and **S** (Section 1.2.1). In the current – *third* version of this work time budgets for all the algorithms were increased roughly 10 times, as described in Section 1.2.2 for more correct comparison of *metaheuristics* against all the other algorithms.

3.3 Efficiency

I introduce two metrics of efficiency which both characterize how fast an algorithm can reduce relative or normalized cost over time. This characteristic – the relative or normalized difference between costs at two consecutive timepoints – over time, should be then calculated as a single number, which is area under the curve divided by time. To visualise the use of this characteristic, one can compare the simple heuristic algorithm which can immediately decrease the cost in much shorter time against a random algorithm, which decrease-time curve would look more like noisy straight line.

$$rel_eff_T := \frac{1}{T} \sum_{i=0}^L [t_{i+1} - t_i] \cdot rel_cost(\pi_i) \equiv \frac{1}{T \cdot c_{best}} \sum_{i=0}^L [t_{i+1} - t_i] \cdot c(\pi_i) - 1$$

The formula for *normalized efficiency* is similar but subtracts normalized best cost

$$\begin{aligned} norm_eff_T &:= \frac{1}{T} \sum_{i=0}^L [t_{i+1} - t_i] \cdot norm_cost(\pi_{t_i}) - norm_cost(\pi_{best}) \equiv \\ &\equiv \frac{1}{T \cdot \|A\|_F \|B\|_F} \sum_{i=0}^L [t_{i+1} - t_i] \cdot c(\pi_i) - \frac{c_{best}}{\|A\|_F \|B\|_F} \end{aligned}$$

where π_i is a solution (permutation) found and accepted at time t_i ; $t_0 = 0$, $c(\pi_0)$ – cost of the initial (random) permutation, $T = \sum_{i=0}^{L+1} [t_i - t_{i-1}] \equiv t_{L+1}$ – time interval between the beginning and the end of a single experiment (run) of an algorithm for a given problem instance, and t_i is a time moment when the cost changes (and this change is accepted), L – number of such changes during the experiment, t_L – time of the last cost change, $t_{L+1} \equiv T$ – time of the end of an experiment (or time

budget). Since both metrics indicate how fast the cost decreases, *the lower the value of these metrics the better*.

It should be noted that this metric allows comparison only when time intervals are identical or very close, and thus time budgets should be the same for such comparison. *While the main quality and runtime comparisons in Sections 3.1 , 3.2 use a balanced mean budget, the efficiency analysis explicitly benchmarks the speed of cost reduction under two contrasting scenarios: a **strict budget** (median runtime of **H**) to evaluate rapid early convergence, and a **generous budget** (median runtime of **S**) to evaluate long-term improvement potential (Section 1.2.1).* Below are results for these two time budgets.

For visual comparison the second metric is more suitable since it keeps plots strictly below 1 on y axis.

Figures 4, 5, 6 and 7 compare efficiency of all the algorithms given “strict” (determined by running time of **H**) and “generous” (determined by running time of **S**) time budgets. In the second efficiency analysis **H** is given for reference only, and excluded from analysis because the proposed efficiency metrics require an algorithm to operate on the whole time interval, while **H** always terminates early and cannot show further improvements, while demonstrating inferior results almost always.

The figures demonstrate superiority of **G** over the rest of the algorithms in both time budget settings, even though “strict” time budget is determined by running time of **H**, and “generous” time budget is determined by running time of **S**, making these conditions sub-optimal for **G** and favourable for **S**.

As an exception, **LAMSA** outperforms all the algorithms in the “strict” setting on the problem *tai256c*, suggesting that extra complexity of this algorithm is effectively deployed in this context.

One can compare efficiency in strict time budget setting against generous setting. We can schematically conclude, that in general efficiency of algorithms follows the following logic (the lower the better the efficiency):

- Strict time budget: **LAMSA** < **SA** < **TS**, *starting from wil100*.
- Generous time budget: **G** $\approx$ **TS** < **S** < **SA** < **LAMSA** < **RW** < **RS** < **H**, *except for esc32e*.

Another comparison can be done in terms of efficiency improvement (compare median change in Figures 5, 7) when transitioning from “strict” to “generous” setting (the lower the better), except for *lipa80b*, where **TS** showed the best improvement:

- **TS** < **S** < **SA** < **LAMSA** < **RW** $\approx$ **RS** < **H**

Identified relationships show that **TS** effectively manages to allocate increased time budgets, suggesting that tuning simulated annealing hyperparameters towards more intensification could be beneficial.

3.4 Analysis of Budget Scaling and Metaheuristic Dynamics

3.4.1 Multi-restarts and increased time budgets

To better analyze and compare efficiency of the algorithms, the following change was done to the experiment setup in the current - *third* version of this study, as described in Section 1.2.2: **S** and **G** were restarted internally 10 times, and **H**, being the fastest of them, was restarted 100 times. Thus time required to run these three algorithms increased 10 or 100 times accordingly. All the other algorithms were given time budgets based on the running time of the former three (*Note: **RW** restarts 10 times, each restart takes one tenth of the budget*). This ensures proper comparison of the metaheuristics and all the other algorithms:

- all algorithms operate under equivalent wall-clock time budgets
- *metaheuristics* are given sufficient time to progress through their full exploration-exploitation cycles
- established exploration parity between *multi-starts local search* algorithms and *metaheuristics*

3.4.2 Hyperparameters of metaheuristics

Selecting proper hyperparameters for *metaheuristics* relies heavily on the provided time budget: strict time budgets would dictate choosing more exploitation-oriented values, while generous time budgets would encourage using more exploration-oriented ones. The previous, *second* version of the current study relied on the stricter budgets, while in the *third* version time budgets for *metaheuristics* increased roughly 10 times (see Figure 3). Thus one would suggest that adjusting diversification behaviour could be beneficial.

Though **SA** determines some hyperparameters based on the time budget (e.g. number of cooling schedule steps), along with **LAMSA**, while **TS** does not, several modifications of the remaining hyperparameters were checked in order to determine whether these *metaheuristics* can benefit more with the increased time budgets:

- **SA**: increased Markov chain factor by 2-2.5, initial acceptance rate was increased up to 0.98
- **TS**: tabu tenure increased twice, size of the elite candidate moves increased twice.

However no significant, indistinguishable from random fluctuations, improvement over baseline hyperparameters in terms of quality or efficiency was found during these experiments. Thus all the conclusions and reported figures in the current section rely on the old values of the hyperparameters, which were defined in the *second* version of this work in Section 8.

3.4.3 Summary of Efficiency Observations

1. Minimal time budgets place **TS** into unfair conditions compared to other algorithms (compare Figure 5 and 7: **TS** clearly underperforms due to lack of time), justifying the previously defined fear that *metaheuristics* require more time to make use of their complexity. One can conclude that this configuration of the efficiency analysis is rather irrelevant, and the findings below will relate generous time budgets mainly.
2. Increased 10 times budgets not only helped *metaheuristics* to employ their complex behaviour more efficiently, but also significantly improved results of **S** and **G**: compare Figure 6 in the current and in the previous (*second*) version of the report: for instance, in the problems *tai256c* and *tho150* **TS** now shows nearly the best results and beats **G**, while all the algorithms show decreased (improved) values of the efficiency; in *will100* and *tai150b* distributions of the efficiency values of **S**, **SA**, **LAMSA** and **TS** noticeably moved towards zero, but **G** still outperforms them (while **TS** shows similar or better results: see 7), suggesting that in these two problems **G** performs better if time budgets are a bit stricter, **TS** shows its best only given large time budget. Examination of Normalized cost reveals similar observations, see Figure 1 – in problems *lipa80b*, *tai80b* *metaheuristics* shorten difference with **G** and **S**, while the latter moved closer to the best solution. In the problem *lipa80b* **TS** clearly wins both in terms of efficiency (given generous time budgets, see Figure 6) and normalized cost (Figure 2).
3. These findings suggest that basically no drastically significant changes happened with increased time budgets: relationships between the figures mainly remain the same (compare Figure 7 in the second and the current version of the report). The main fear that the complexity of metaheuristics was wasted due to strict time budgets finds its justification in the case of “strict” time budgets, where metaheuristics lose clearly due to extra complexity. Additionally, it helped **TS** to improve positions in the “generous” setting, though **G** still mainly outperforms all the algorithms in terms of efficiency.
4. Increased approximately 10 times budgets generally gave an equal advantage to all the algorithms. Thus many findings in the Sections 3.7, 9, 9.3 remain the same as in the previous, *second* version of this work. Probably, more careful tuning of the hyperparameters, adjusting them specifically to increased time budgets, could yield better results, but this is beyond the scope of this work.
5. At the same time transition from the “strict” time budget setting to the “generous” one reveals significant improvement in the terms of efficiency for all the algorithms (compare Figures 6 and 4), but the most improvement happened to **TS**, **S**, **SA** and **LAMSA**, as identified in Section 3.3.

Boxplot of Relative Efficiency of solutions found by different methods given minimal time budgets determined by the Heuristic local search, by problem instance

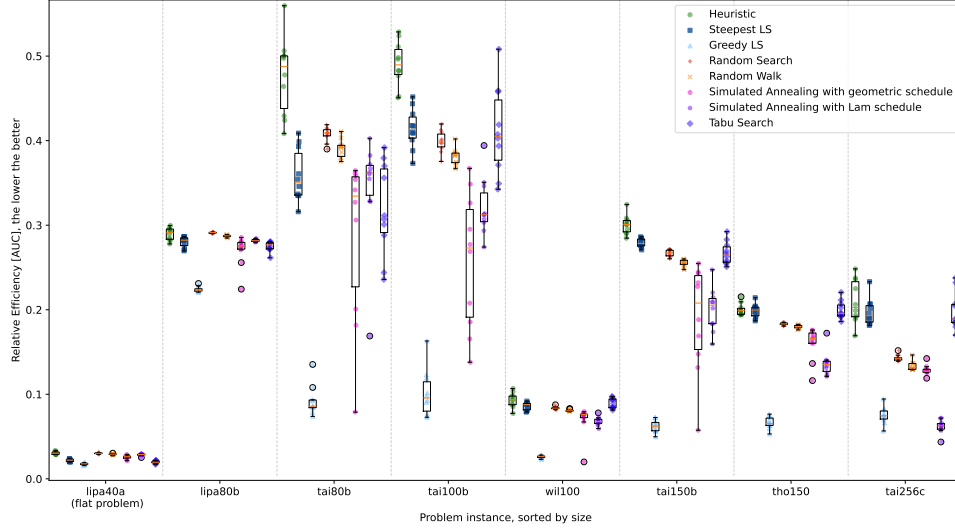


Figure 4: (truncated) Relative Efficiency for minimal time budgets

Boxplot of Normalized Efficiency of solutions found by different methods given minimal time budgets determined by the Heuristic local search, by problem instance

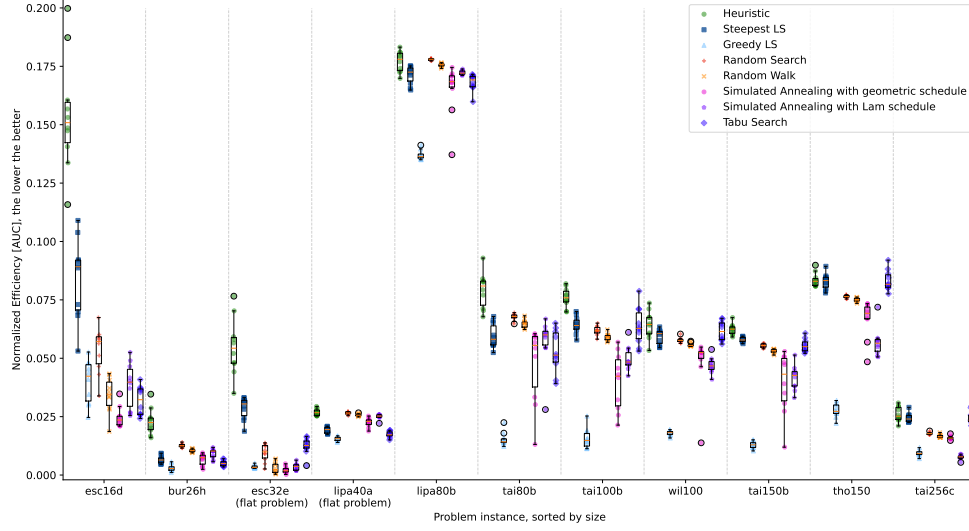


Figure 5: Normalized Efficiency for minimal time budgets

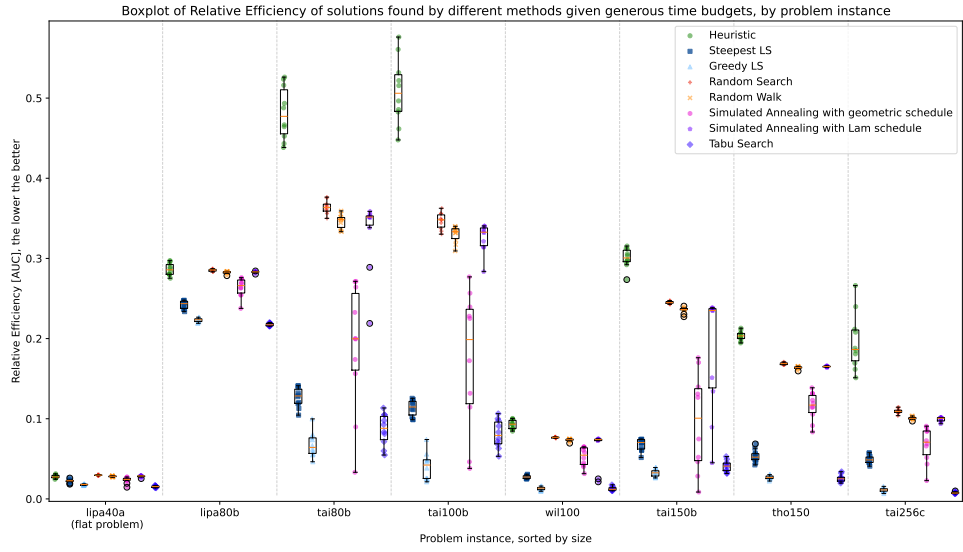


Figure 6: (truncated) Relative Efficiency for generous time budgets

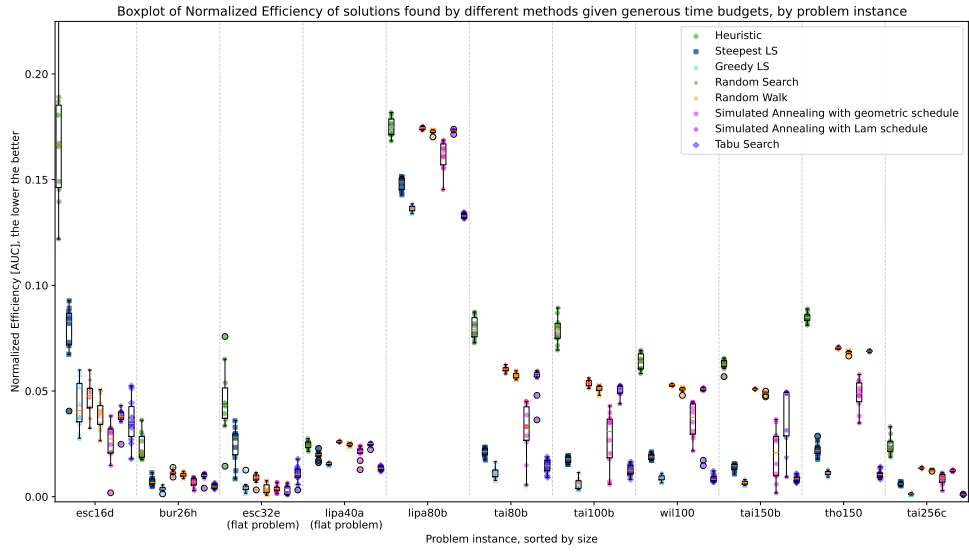


Figure 7: Normalized Efficiency for generous time budgets

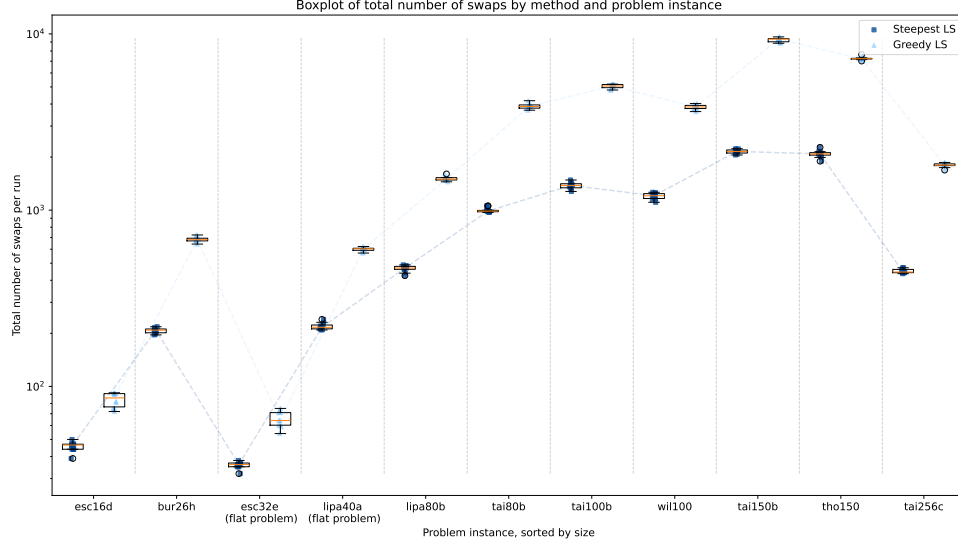


Figure 8: Total number of swaps – greedy and steepest local search

3.5 Total Number of Algorithm Steps

The **step** is determined as a swap of two elements in the current solution (permutation).

Figure 8 shows that **G** performs much more swaps than **S**, and thus shows that exploratory behaviour (driven by choices that are not necessarily locally optimal) is beneficial when solving all the problems, given the fact that **G** also showed superior quality. Figure 9 illustrates that **SA** and **LAMSA** behave surprisingly similar to **RW**, while **TS** performs similar to **S** number of swaps: unlike **G**, both **TS** and **S** tend to carefully examine all the neighbours and execute either the most or enough promising moves.

3.6 Total number of evaluated solutions

An evaluated (fully or partially) solution is a solution (permutation) for which a cost value was obtained.

As illustrated in Figure 10, despite the visual discrepancy between total number of evaluated solutions between **S** and **G**, the former evaluates all the solutions in a current neighbourhood, while **G** evaluates solutions on a much more distant scale given the same processing time, though in more stochastic manner. At the same time **G** and **SA**, **LAMSA** seem to have evaluated the same number of solutions (with **TS** evaluating a slightly smaller number), while the time spent by **G** is lower, which aligns well with the observations from efficiency plots.

RS seems to evaluate orders of magnitude lower number of solutions due to the expensive full evaluations of the objective function (cost) of each random solution.

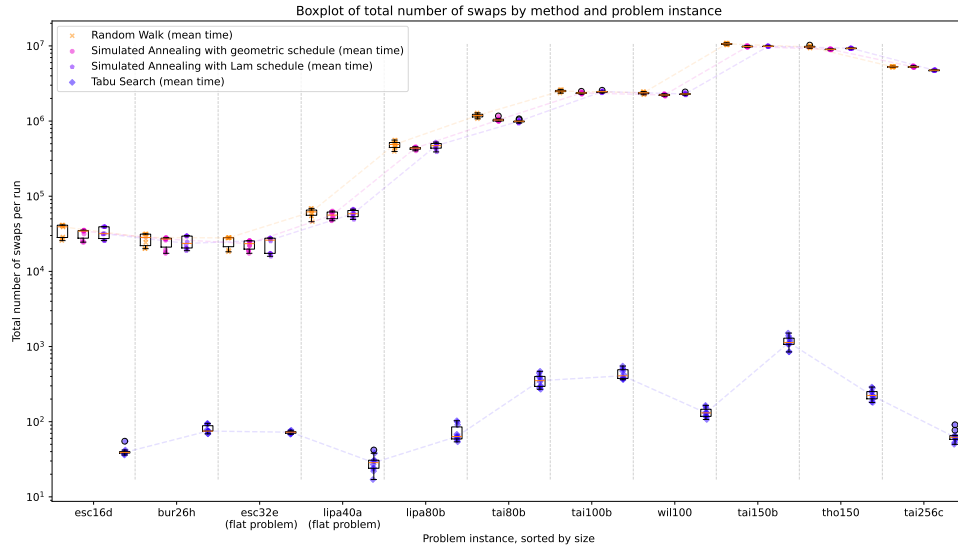


Figure 9: Total number of swaps – random walk, simulated annealing and tabu search

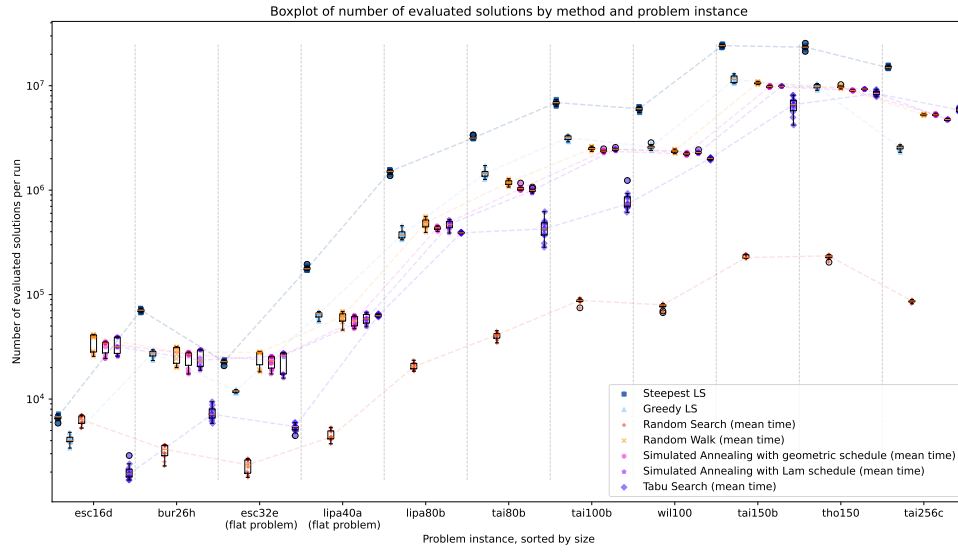


Figure 10: Number of evaluated solutions

3.7 Conclusions

Given all the evidence demonstrated so far, one can conclude the following:

1. Exploratory behaviour is beneficial when solving QAP.
2. Prior information should be utilized to make exploration more efficient: pure **RS** is absolutely inefficient in solving QAP.
3. Surprisingly, **SA** outperforms **LAMSA**, which could be possibly explained by the fact that my implementation of **SA** tends to discover as many available “temperature-tiers” as possible, based on the given time budget, compared against **LAMSA** which spends most of its time with acceptance rate ≈ 0.44 .
4. **TS** outperforms **SA** in terms of efficiency, while **SA** in total yields better quality of solutions. Probably, this can be explained by the fact that former one utilizes memory to store promising moves.
5. Essentially, **G** and **SA**, **LAMSA** perform similar number of solution evaluations, while **TS** performs a bit less.
6. **TS** behaves extremely similar to **S**, while **SA** and **LAMSA** – to **RW**. This follows not only from the observations, but also from the way these algorithms are defined.
7. **In many problems SA** sometimes (few times out of 10 runs) tends to find the best among all the other algorithms solutions (see Figures 1, 2). Thus running this algorithm multiple times in parallel is a highly efficient strategy to solve many problems.
8. **In general**, greedy strategy **G** outperforms all the other algorithms in terms of efficiency, and almost always finds the best (median) solution compared to its competitors. It utilizes time, spent by complex metaheuristics (**SA**, **LAMSA**, **TS**) which tend to optimize the walk across the space, to explore new and new solutions, improving move with minimal computational effort. These *metaheuristics* are not able to utilize geometry of solution spaces any better than a simple greedy choice. One can think of this as follows: in all the problems examined, the best strategy is “don’t think much, just move”. *Note: Probably, tai256c is the only problem, where another approach is more beneficial if time budget is strict.*
9. In the previous report, **TS** was severely handicapped by its initialization overhead. With 10 times increased “generous” budgets, **TS** closes the gap with **G** across all instances and strictly outperforms **G** on *lipa80b*, *tho150*, and *tai256c*. This suggests that in general, among other *metaheuristics* **TS** especially required increased budgets to make use of its complex mechanics. This extra “thinking time” pays off on structured instances and allows to deploy its intensification oriented mechanics (proved to be beneficial as showed by **G**), while more stochastic nature of **SA** prevents it from generating more accurate trajectories.
10. **Landscape structure and exploration mechanism** directly influence performance of the algorithms:
 - (a) in unstructured, **uniform** instances (*tai-x-b* series) the landscape has no global funnel (see Section 4 - low Spearman correlation), **G** excels because restarting multiple times from different starting points allows to discover different basins.
 - (b) in structured, **skewed** problems (*tai256c*, *lipa80b*, *tho150* with high *Fisher-Pearson* skewness – see *Appendix*), **TS** wins under “generous” budgets because its memory and elite candidates navigate through plateaus and saddles rather than abandoning the search trajectory.
11. Probably, more careful hyperparameter tuning in **SA**, **LAMSA** and **TS** would dramatically improve their performance and final costs of solutions, compared to **G**. However, rather than relying on manual, ad-hoc hyperparameter tuning for each individual problem instance, this work employs adaptive, budget-aware parameter derivation (e.g., analytical derivation of α' from the time budget instead of selecting a value from the given interval, initial temperature

T_0 calibrated to average deteriorating moves $\bar{\Delta}_{c_{\text{worse}}}$ in **SA**). This ensures robust, unbiased, and reproducible comparisons across all problem scales. At the same time looking for a unified strategy for hyperparameters definitions, which would yield much better results, requires careful examination of additional statistical and topological properties of QAP instances on the whole, which is out of scope of the current work.

12. Size of a problem matters. Many findings are mostly valid for “big” problems (with size ≥ 80). For small problems different algorithms show slightly more variability probably due to noticeable difference in operational costs or concentrically lesser difficulty of “small” problems.

4 Discovering the Structure of the Search Space and the Optimized Function

The following instances were chosen for the analysis:

1. *lipa80b*, since all the methods demonstrated sub-optimal solutions compared to the optimal solution (which is equal to the identity permutation).
2. *tai100b*, due to noticeable variability in relative cost.

The other problems are not very interesting due to the lack of variability in final quality metrics. 1000 independent runs were performed of **G** and **S** for each of the problems above.

The following Figures 12 and 11 demonstrate relationships between initial and final quality. For this purpose *relative* and *normalised* cost can be used interchangeably when analyzing particular problems since every point on one plot can be linearly translated to the other plot. *This rule doesn't hold when points for multiple problems are plotted on the same graph.*

Figure 11 demonstrates both clusters corresponding to these two problems, together. Figure 12 shows the same clusters in *normalized cost* terms. Spearman correlations are provided per each method and per each instance.

In both cases clouds of points were obtained without any visible structure. There is an isolated point (*outlier*) in *lipa80b* problem, which is equal to the optimal solution. This happened few times when thousands of runs were performed. The distinct feature of these clouds is that the cloud corresponding to **G** lies consistently lower, which supports the earlier conclusions that **G** performs better than **S**. Spearman correlations show close to 0 values (i.e. within $(-0.05; 0.05)$ interval), meaning that there is no direct, monotonic “better in the beginning – better at the end” relationship between initial and final costs. Local search can map very different starts to very similar final costs. So runs with bad initial ranks and good initial ranks end up mixed in final ranks. Possible explanations of the observed low Spearman correlation are determined by landscape of the problems:

1. Multiple local minima/basins with different depths.
2. Large basins around some minima.
3. Plateaus or many near-ties in final quality.

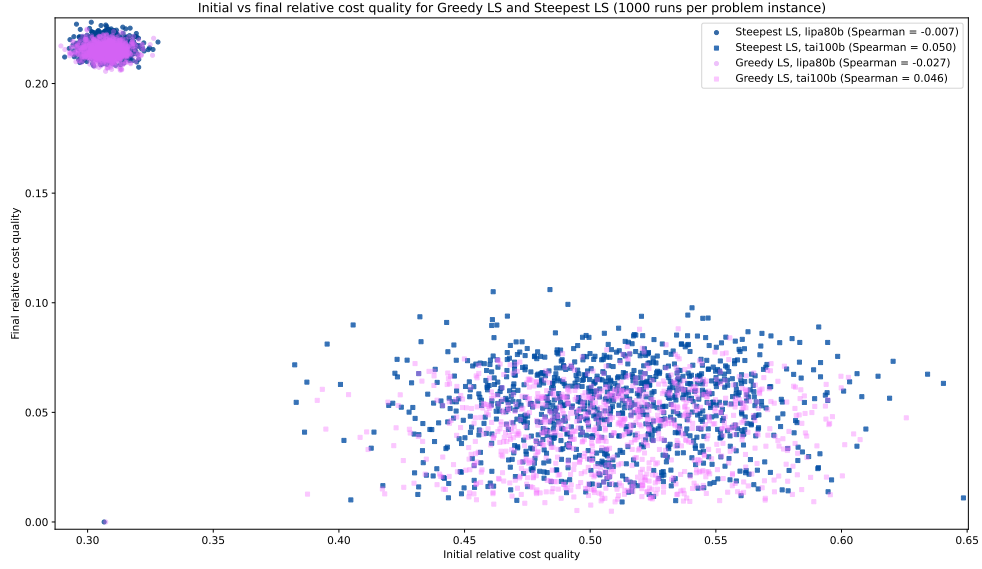
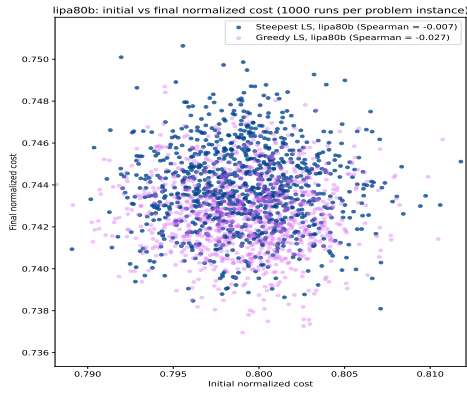
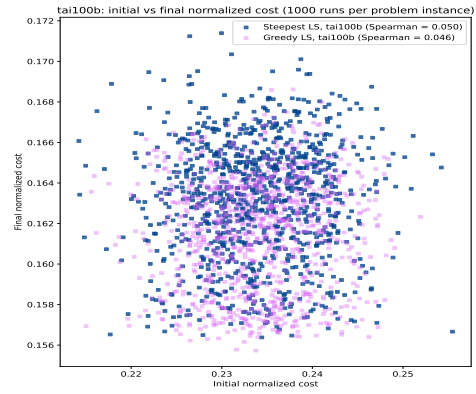


Figure 11: Initial vs Final Relative cost of 1000 runs of **G** and **S**



(a) *lipa80b* (truncated, top cluster)



(b) *tai100b* (bottom cluster)

Figure 12: Initial vs Final Normalized cost of 1000 runs of **G** and **S**

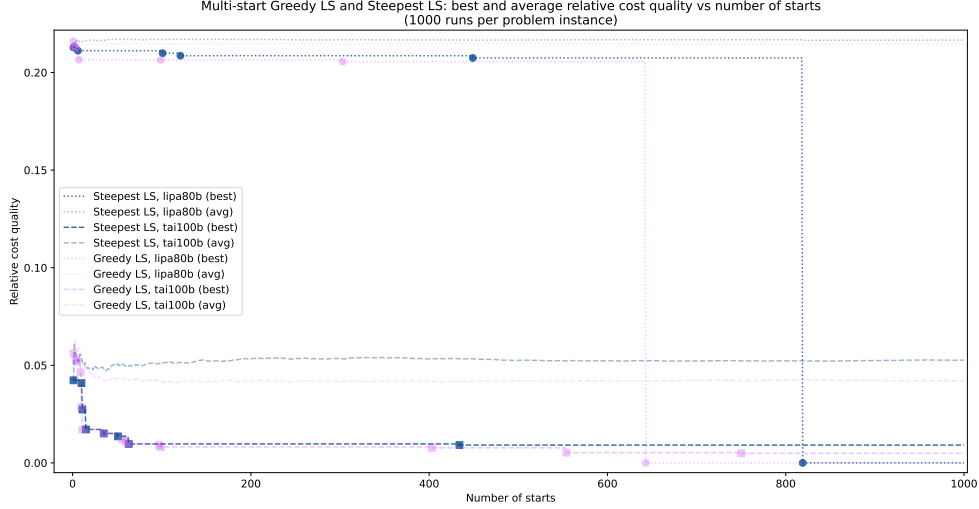


Figure 13: Best and Average relative cost plot for 1000 independent runs

5 Discovering the Preferred Number of Repetitions of LS

In this experiment, the horizontal axis in Figure 13 is the number of independent starts $k=0..1000$, and for each k two values are plotted: *best-so-far relative cost quality* and *average-so-far relative cost quality* over the first k starts.

As was mentioned before, *lipa80b* has a significant improvement in the cost once during a few thousands (sometimes more often) restarts, and thus it makes sense to repeat **S** few thousands times (then the algorithm will find an optimal solution). At the same time *tai100b* demonstrates better improvement during one to few hundreds starts. Based on the visual characteristics of the **G** curve, it's better to stop early at $t = 150$ as plateau-based practical cutoff.

Hence, repeating LS is worthwhile, but the preferred number of restarts is instance- and algorithm-dependent: moderate budget is proposed for *tai100b*, and larger budget for *lipa80b* when chasing rare best-case gains.

6 Objective Assessment of the Similarity of Locally Optimal Solutions Found

Cayley similarity between two permutations π_1 and π_2 is defined through *Cayley distance* which is the minimum number of swaps necessary to obtain π_2 from π_1 :

$$\begin{aligned} \text{Cayley_sim}(\pi_1, \pi_2) &:= 1 - \text{Normalized Cayley distance}(\pi_1, \pi_2) = \\ &= 1 - \frac{1}{N-1} \min |\{k : s_k = \text{swap}_{i_k j_k}, \pi_2 = (\prod_k s_k) \cdot \pi_1\}| \end{aligned}$$

This similarity measure is proposed due to the natural definition of a metric on the space of permutations of length N (*permutohedron*) as it reflects how far are two permutations in the solutions space.

As a valid alternative, another possible metric could be used to introduce similarity between permutations – Hamming distance.

Since both *Cayley* and *Hamming distances* are valid mathematical metrics, these similarity functions, for a given problem size N , naturally map distances into similarities.

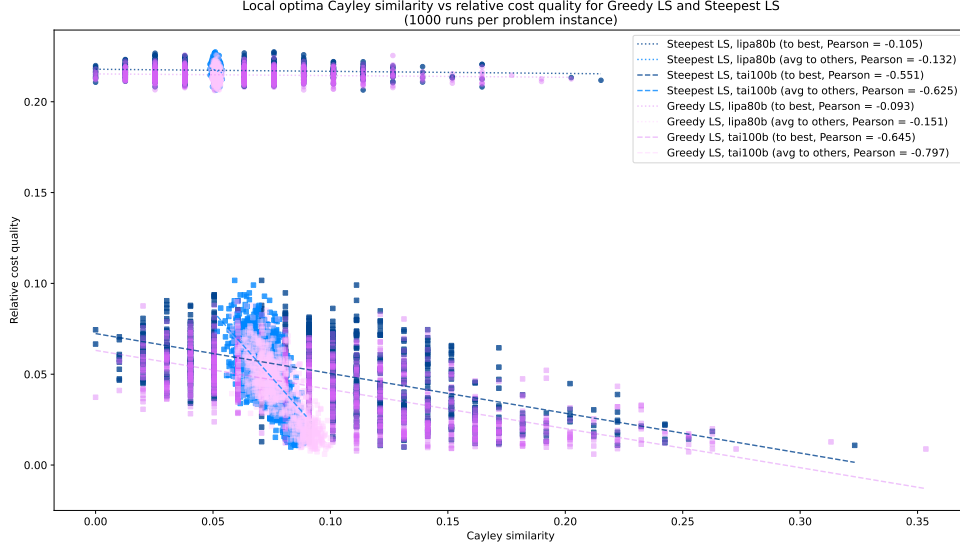
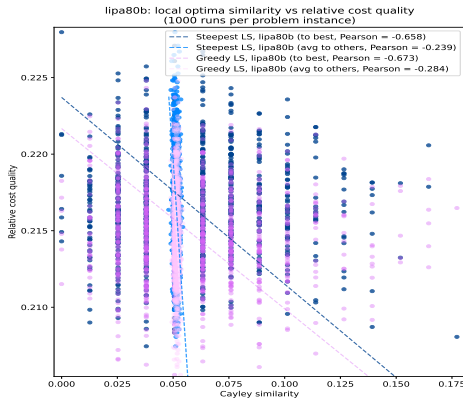


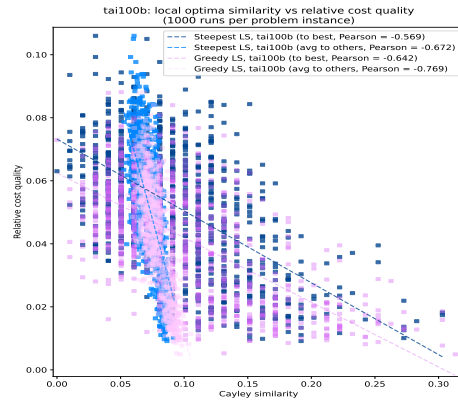
Figure 14: Cayley similarity vs. Relative Cost on data without outlier in *lipa80b*

The following two graphs demonstrate relationship between Cayley similarity and *relative cost* (as it is more informative for multiple problems plots):

1. Figure 14: data without an outlier, obtained by new 1000 restarts of algorithms. The plot demonstrates two clusters of points which correspond to *lipa80b* and *tai100b* problems. The same clusters, obtained during another round of experiments with outliers in *lipa80b*, are depicted on the next figure.
2. Figure 15: data with a found outlier in *lipa80b*.



(a) *lipa80b* (outliers truncated, top cluster)



(b) *tai100b* (bottom cluster)

Figure 15: Cayley similarity vs. Relative Cost on data with outliers in *lipa80b*

The figures demonstrate that there is from *moderate to strong linear relationship* between the similarity of a found solution (local optima) and the quality in *tai100b* problem. However, no such conclusion can be made in the case of *lipa80b* as this artificial problem has only one distinct global optima (isolated point on the plot 11, which corresponds to the identity permutation). At the same time *tai100b* seems to have a lot of local optima with cost close to the best known one, and the closer these local optima to the best known solution, the lower the cost.

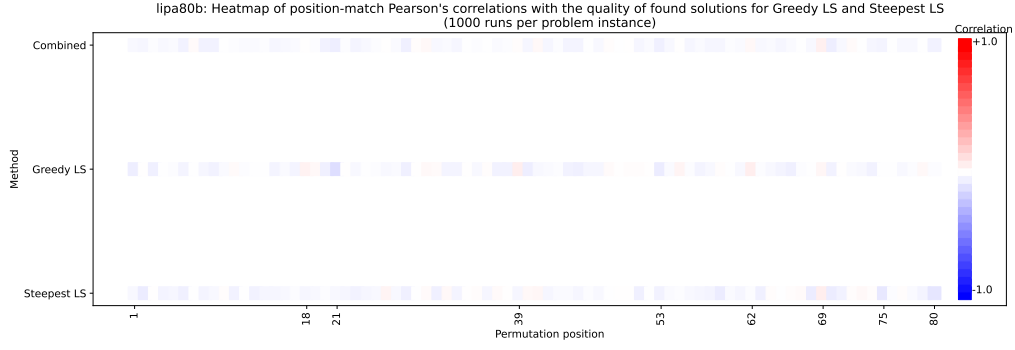


Figure 16: *lipa80b*: Dependency of position-matches with the best known solution and the quality of solutions

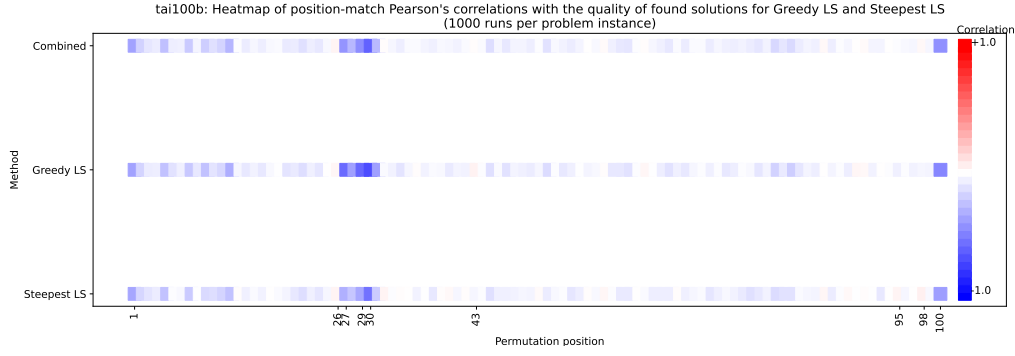


Figure 17: *tai100b*: Dependency of position-matches with the best known solution and the quality of solutions

The following heatmaps show relationships between separate positions, matched with the positions of the best known solution, and the quality of the solution (same for both *relative cost* and *normalized cost* quality metrics).

Hence *lipa80b* has no particular position in the solution which would be responsible for a significant cost reduction, however there are few such positions in *tai100b* (namely, almost all the positions from 1 to 31 have low negative correlation with the cost, and positions 27, 29, 30, and 100 have moderate negative correlation with the cost).

Note: The best-known solutions from *.sln* files use 1-based indices, which were converted to 0-based before computing similarity – when loading data into *Problem* class, and then passing the solution into *Permutation* structure.

7 Heuristic Algorithm for QAP

The heuristic algorithm (**H**) is defined as follows:

1. For the current row, find the largest element in the first matrix and the smallest element in the second matrix.
2. Swap the permutation indices corresponding to the columns containing these elements, but only if the swap improves the objective value.
3. Move to the next row and repeat the search starting from the diagonal element of that row, i.e. ignoring columns that have already been considered. This restricts later swaps from using earlier column indices and helps preserve previously matched positions.
4. Repeat steps **1–3** until all rows except the last one have been processed.
5. Repeat steps **1–4** until no improving moves are found, or until the imposed iteration or time limit is reached.
6. **Modification** added in the third version of the study: sample a new permutation and repeat steps 1-5, updating the best solution achieved so far and associated statistics whenever the best cost is improved. This step is repeated 100 times specifically in the experiments for Section 3, and omitted in the complexity analysis below.

The complexity of one outer pass is $O(n^2)$. For each of the $n - 1$ rows, the algorithm searches linearly through the remaining part of the row and evaluates the candidate swap using the incremental cost function, whose complexity is $O(n)$. Therefore, if the number of outer passes is denoted by I , the total complexity is $O(I \cdot n^2)$.

If a maximum number of iterations M is imposed, then $I \leq M$ and the complexity becomes $O(M \cdot n^2)$. Without such a limit, the algorithm still terminates because every accepted swap strictly decreases the objective value and the number of permutations is finite. A loose worst-case bound is therefore factorial, although this bound is not usually observed in practice.

An important property of this heuristic is that the candidate swaps are fixed by the input matrices. Since the matrices do not change during the search, the indices of the selected largest and smallest elements are the same in every outer pass. Thus, the algorithm repeatedly considers at most $n - 1$ different swap types, instead of the full neighbourhood of $\frac{n(n-1)}{2}$ possible swaps. These pairs can be cached, which reduces redundant computation and improves the constant factor, although the asymptotic complexity of one pass remains $O(n^2)$ because of the incremental cost evaluations.

The algorithm could be further extended by matching whole rows or columns rather than individual matrix elements.

8 Simulated Annealing and Tabu Search

This section describes implementation of *Simulated Annealing* (**SA**, **LAMSA**) and *Tabu Search* (**TS**) added to the second version of the report. As in the case of random methods (**RS**, **RW**), these methods were provided the same time budgets in the corresponding experiments in Section 3.

8.1 Simulated Annealing

Simulated annealing is a *local search based metaheuristic*, specifically designed to escape local optima by accepting worse moves with a probability which decreases over time. The algorithm consists of the following steps:

1. Start with initial solution (permutation) $\pi = \pi_0$, $t = 0$
2. Generate a new random solution π' from the neighbourhood $N(\pi_t)$
3. Accept π' if $c(\pi') \leq c(\pi_t)$ and update the best cost and solution, if necessary. Otherwise accept with the probability (*acceptance rate*):

$$p_{accept}^t = \exp\left(\frac{-\Delta c}{T_t}\right)$$

where $\Delta c := c(\pi') - c(\pi_t)$

4. $\pi_{t+1} := \pi'$
5. Recalculate temperature T_t according to the *cooling schedule* $\rightarrow T_{t+1}$
6. Timestamp update: $t := t + 1$. Repeat from the step **2** until stopping criteria is met (that is time elapsed or no improvements condition is met).

8.1.1 Stopping criterion

Algorithm terminates if running time exceeds the specified limit or number of steps in the cooling schedule exceeded.

8.1.2 Hyperparameters

Simulated annealing requires the following hyperparameters to be defined:

- Initial temperature, T_0
- Cooling rate, α
- Length of Markov chain, L
- Number of steps of the cooling schedule, N_{steps}

The following values were used: $L := |N(\pi)|$, i.e. the size of the neighbourhood of π , where π is the current solution (permutation); cooling rate is determined adaptively in **LAMSA**, initial cooling rate α_0 is set to 0.85 for **SA**; number of steps N_{steps} is defined as *time budget divided by average time necessary to complete one cooling schedule step* (one step is exactly L iterations simulated annealing, during which temperature is constant) – recalculated every L steps for better precision; initial temperature is determined by the following formula:

$$T_0 := \frac{-\bar{\Delta}c_{worse}}{\ln p_{accept}^0} = \frac{-\bar{\Delta}c_{worse}}{\ln 0.95}$$

where p_{accept}^0 is initial *acceptance rate of a deteriorating move* set to 0.95.

8.1.3 Geometric schedule

For a fixed initial temperature T_0 and Markov chain factor L , temperature is updated as follows

$$T_{t+1} := T_0 \cdot \alpha^{\lfloor t/L \rfloor}$$

That is, temperature is decreased by factor α each L steps.

Since for different problems acceptance rate was decreasing either too fast (final acceptance rate drops below $1e-5$) or too slow (final acceptance rate $\approx p_{accept}^0$), cooling rate was recalculated each L steps based on the formula:

$$\alpha' := N_{steps}^{-1} \sqrt{\frac{\bar{\Delta}c_{worse}}{T_0 \ln \frac{p_{accept}^0}{p_{target}} + \bar{\Delta}c_{worse}}} = N_{steps}^{-1} \sqrt{\frac{\bar{\Delta}c_{worse}}{\frac{-\bar{\Delta}c_{worse}}{\ln 0.95} \ln 1000 + \bar{\Delta}c_{worse}}} = N_{steps}^{-1} \sqrt{\frac{1}{\frac{-\ln 1000}{\ln 0.95} + 1}}$$

where p_{target} is *target acceptance rate*, set to be exactly 1000 times smaller than p_{accept}^0 . Pay attention, that the only variable in this formula is N_{steps} , and thus α' is rather a periodically updated estimate of the hyperparameter α .

Note: The derivation of this formula is trivial and is left as an exercise for the reader, as it follows immediately from the desired 1000-fold decrease in the acceptance rate:

$$\frac{p_{accept}^0}{p_{target}} := 1000$$

In the formulas above, $\bar{\Delta}c_{worse}$ is an average cost increase of sampled *deteriorating* moves from the initial permutation, estimated at the beginning of an algorithm.

The idea behind this *alpha-schedule* is to allow simulated annealing to perform evaluation of possible solutions on different levels of temperature, reliably moving from exploratory to exploitative behaviour, which is difficult to achieve with fixed cooling rate. Defining alpha in this way, the two extreme yet common issues in the temperature scheduling are resolved: temperature drops either too slowly (if time budget is too strict for the given problem) or too fast (if time budget is generous).

Thus, if time budget is stringent and only few cooling schedule steps will be performed (e.g. $N_{steps} < 15$), cooling rate α can drop below 0.75 – in my experiments, this is usually the case for the problems of small size. On the contrary, if time budget is generous, number of steps can be big enough to keep cooling rate $\alpha \approx 0.95$ (which is much higher than the initial estimate of $\alpha := \alpha_0 \equiv 0.85$), hence fostering exploration.

An alternative approach to recalculate α' could be to set T_0 to the temperature during previous stage of the schedule and use estimated number of remaining steps instead of N_{steps} , but the dynamic of the cooling schedule would look more stochastic.

8.1.4 Lam Schedule

LAMSA is an adaptive simulated annealing approach that dynamically adjusts the temperature based on the measured acceptance rate of deteriorating moves, acting much like a thermostat to maintain thermal equilibrium. Unlike standard geometric schedule, it follows a specific ‘Lam Curve’ for target probabilities:

1. first 15% of iterations: fast drop from 1.0 to 0.44;
2. next 50% of time: the “plateau” – stay at probability of 0.44, for efficient optimization;
3. final 35%: quick cooling down to 0.0 probability.

The algorithm utilizes a ‘proportional controller’ at the end of each Markov chain to adjust the temperature up or down based on the actual rate of acceptance of bad moves, ensuring the search remains at the target probability even as the ruggedness of the fitness landscape changes.

Precise description of simulated annealing with Lam schedule is given in [6] and pseudocode can be found in [7].

8.2 Tabu Search

Tabu search (**TS**) is a *local search based metaheuristic*, which intelligently explores the solution space by utilizing memory. The following features define the architecture of **TS**:

- **Tabu list** – is a recency-based memory structure that stores recently executed moves. It forbids the algorithm from selecting the exact same pair of indices (i, j) for a specified number of iterations (**tabu tenure**), thereby preventing the search trajectory from immediately reversing its previous steps and getting trapped in 2-cycles.
- **Worsening moves** are allowed if all improving moves are tabu.
- **Aspiration criteria** – rules that allow the algorithm to temporarily override tabu restrictions if a forbidden move leads to a highly promising solution. The simplest and widely used aspiration criterion was used – *if a move is tabu but improves the best cost achieved so far, it is accepted*.

8.2.1 Elite candidate list

This is short-term-memory-like structure which defines moves of a high quality that are considered to be executed during the course of the algorithm. The list of elite moves is constructed by examining all the moves in the neighbourhood of the current solution (permutation p) [8]. Next at each subsequent iteration the best non-tabu move is chosen from the list and executed. This procedure is repeated until quality of such moves falls below a predefined *threshold* or a given number of iterations elapsed (elite candidate list *lifespan*). Then the list is rebuilt anew.

Constructing such a structure aims to avoid expensive computations by defining promising moves in advance [9]. But this idea relies on the fact that quality of elite moves will not change much during the lifespan of this list. However, this assumption fails in the case where cost function landscape is heavily rugged and cost burden is distributed uniformly over all the nodes (facilities) in both matrices. And conversely, this assumption holds when many pairs behave independently, and the elite moves do not involve highly scored intermediate interconnections. These considerations convey the suggestion that *problem skewness* (which can be defined as maximum skewness of data in both problem’s matrices, by absolute value) directly correlates with the reliability of this assumption.

A further improvement of this idea could be to dynamically adjust the list size sz based on the rate of invalidated moves once the quality of the moves drops beyond a threshold. For example, if **TS** reconstructs the list almost every iteration as this happens in the problem *tai256c*, the list size, set to 6, could be halved.

8.2.2 Hyperparameters

TS requires the following hyperparameters to be defined:

- Elite candidate list lifespan, Lf
- Elite candidate list size, sz
- Tabu tenure, τ
- delta drift relative threshold, th

Lf defines the number of iterations elite candidate list is considered to be valid.

Since the lifespan highly depends on ruggedness and uniformity of the data in the problem matrices, in order to properly set this value, Lf was heuristically defined as scaling proportionally with the problem skewness’ absolute value $|\gamma|$ and hard-capped by $N^{2/3}$:

$$Lf := \max(1, \lfloor f_L \cdot \min(N^{2/3}, \lceil |\gamma| \rceil) \rfloor)$$

where scaling factor $f_L := 1.25$.

The number of the candidates in the list is set heuristically as $sz := 2 \cdot Lf$. Delta drift relative threshold, set to $th := 0.75$ defines how far an actual delta cost can drift from its stale estimate (computed when the elite list was formed) before forcing an early list rebuild. This is an optional hyperparameter (since it’s assumed defining elite candidate list lifespan Lf would be sufficient), which

serves as an alternative to Lf in indicating whether the elite candidate list should be rebuilt – by triggering the following condition:

$$\Delta_{updated} - \Delta_{stale} > th \cdot |\Delta_{stale}|$$

Tabu tenure – the number of iterations an accepted move (a swap) is banned – is defined heuristically as $\tau := \sqrt{N}$.

8.2.3 Stopping criteria

TS stops execution once one of these conditions is satisfied:

- The total elapsed execution time exceeds the specified limit (time budget).
- Stagnation is detected – the global best solution fails to improve for a consecutive number of iterations, defined heuristically as $2 \cdot N$, where N is the problem size.

8.2.4 The algorithm

The algorithm operates via two nested loops. At each “step” of the outer loop, a fresh elite candidate list is generated based on the current solution (permutation $\mathbf{p}$).

Elite candidates are stored in an automatically sorted container with a limited number of elements sz (wrapping `std::multimap` : $\Delta c \rightarrow move$ with $O(1)$ complexity of finding the best move and removing a move, and $O(\log(sz)) \equiv O(\log N^{2/3})$ complexity of inserting a new entry). This container automatically drops moves of worse quality when maximum number of elements sz is reached and a new, better entry is inserted.

The inner loop sequentially evaluates these candidates. If the top candidate is non-tabu, tabu-expired, or satisfies the aspiration criterion (i.e., strictly improves upon the best cost achieved so far), it is executed, placed to the tabu list and dropped from the elite candidate list. Otherwise, the next elite candidate move is evaluated in the same fashion.

Note: *Tabu status is updated by increasing a timestamp of a move in the tabu list by value τ . Thus moves are not explicitly removed from the tabu list container, rather they lose tabu-status once the current iteration clock is at least τ greater than the initial timestamp.*

If the actual delta of an elite move drifts too far from the stale delta calculated when the list was formed (e.g., exceeding a configured relative threshold th), the elite list will immediately be invalidated and reconstructed.

More precisely, the execution sequence is formalized as follows:

1. Start with initial solution (permutation) $\pi = \pi_0$, $t = 0$
2. Inspect all the moves in the neighbourhood $N(\pi_t)$ of the current solution π_t , and populate a pre-empted container E of elite candidate moves using Δc as a key. Bad moves will be discarded automatically upon inserting if maximum number of elements sz reached.
3. Extract the first (the next available) move from E , if available and lifespan Lf of the container E is not exceeded. Otherwise go to step 2 to trigger a scheduled rebuild.
4. If actual $\Delta'c$ of the move π' drifted too much from its stale estimate, go to step 2.
5. Accept π' if $c(\pi') \leq c_{best}^t$ (*aspiration criterion*) or tabu-status of the move expired, and update the best cost and solution, if necessary. If the condition fails, go to step 3 – evaluate the next move in E .
6. Assign $\pi_{t+1} := \pi'$, drop the move from E and place it in the tabu list to update its tabu expiration timestamp.
7. Timestamp update: $t := t + 1$.
8. Evaluate the next move in E – repeat steps starting from 3.
9. Repeat from step 2 until stopping criteria is met (that is, time elapsed or the best solution has not improved for a specified number of iterations).

9 Conclusions

9.1 General

- **QAP is NP-hard with a rugged landscape.** Stochastic methods without local guidance (**RS**) or memory (**RW**) are insufficient to find competitive solutions even for moderate-sized instances.
- **Problem structure matters:** it strongly influences algorithm behaviour. “Flat” instances (*lipa40a*, *esc32e*) are degenerate cases where either a near-optimal solution is trivial to construct or cost improvement is structurally infeasible.
- **Exploration Paradigms:**
 - Restart-based fast exploration (**G**) is optimal for flat, uniformly random landscapes (*tai-x*) where local basins are uncorrelated.
 - Continuous trajectory exploration (**TS**, **SA**) is essential for structured instances (*tai256c*, *tho150*) where valley navigation and escaping shallow local minima yield higher quality solutions than random re-initialization.

9.2 Specific

- **Algorithm ranking:**
 - Local search methods (**G**, **S**) significantly outperform **H**, **RS**, **RW** on solution quality across all instances. **G** outperforms **S** in both quality and efficiency under all tested time budgets, attributed to its exploratory behaviour enabling broader landscape coverage per unit time.
 - Under *strict* time budgets, multi-restart **G** dominates all algorithms in both efficiency and quality due to its zero-overhead greedy descent. *Metaheuristics* (**TS**, **SA**, **LAMSA**) are heavily constrained by their initial setup overhead, and their complexity is wasted.
 - Under *generous* time budgets, **TS** closes the gap and matches or surpasses multi-restart **G** on structured instances (*tai256c*, *tho150*, *lipa80b*), demonstrating the value of recency memory and elite candidate lists.
 - Pure random search (**RS**) is ineffective across all budgets, while **SA** achieves top-tier best-case solutions in isolated runs, making it well-suited for parallelized multi-start setups.
- **Initial solution irrelevance:** Near-zero Spearman correlation between initial and final cost (within ± 0.05) across 1000 runs confirms that initial solution quality does not predict final quality, which is consistent with a landscape of many basins of varying depth. This justifies the use of **multi-start strategy** over complex single-trajectory heuristics (especially **SA**).
- **Multi-start strategy:** Repeating LS is worthwhile but the optimal restart budget is instance-dependent:
 - around 200 restarts suffice for **G** on *tai100b*;
 - several thousand are needed for **S** on *lipa80b* to encounter the rare optimal basin.
- **Search space structure (similarity analysis):**
 - *tai100b* exhibits a structured central-massif-like landscape where closer local optima (higher Cayley similarity to the best known) have lower cost (Pearson correlation < -0.6)
 - *lipa80b* has a near-flat landscape with a single isolated global optimum (identity permutation – *needle in a haystack*), with no such correlation. Specific positions 27–30 in *tai100b* account for disproportionate cost reductions.

9.3 Metaheuristic Analysis

Precise definition of the three metaheuristics algorithms – **SA**, **LAMSA**, and **TS** – is given in Section 8.

- Geometric schedule outperforms the adaptive Lam schedule in my experiments. The budget-aware derivation of cooling rate α effectively forced the search through a complete spectrum of “temperature tiers”, ensuring a transition from high-mobility exploration to greedy quenching. In contrast, **LAMSA**’s focus on the “0.44 plateau” might have resulted in the search spending too much of a limited time budget at a single mobility level, potentially missing deeper basins in rugged QAP landscapes.
- **TS** and **S** represent an exploitative class of algorithms that carefully examines neighborhoods to execute high-influence moves. Conversely, **SA**, **LAMSA**, and **RW** form a more stochastic class.
- **TS**’s efficiency is tied to problem skewness. The “elite move” assumption – that a good move remains good for several iterations – holds only in skewed (*tai256c*, *tho150*, *lipa80b*) or sparse problems where dominant flow-distance linkages (tectonic plates) define the landscape, and in practice invalidates very fast. In uniform problems, **TS** behaves mostly like **S** since problem’s ruggedness dictates an elite candidates list to be of unit size.
- While these complex metaheuristics aim to optimize the search path through the search space, **G**’s strategy “don’t think much, just move” is often superior in *efficiency* [Section 3.3], with **TS** matching or surpassing **G** in “generous” time setting on structured instances. By reducing “thinking time” per swap, **G** uses computational budget most efficiently for many QAP instances.

10 Challenges Encountered

1. Debugging *Matplotlib-cpp* errors in C++ was difficult and time-consuming.
2. Containerization and dependency setup (C++/Python/plotting) required significant effort.
3. Refactoring core structures introduced temporary performance regressions.
4. *int* overflow caused invalid results (e.g., negative costs), so wider integer types (*int_64t*, *double*) were introduced.
5. Parallelization improved speed but complicated fair benchmarking and determinism.
6. Final figures (with low number of restarts) required strict reproducibility settings (single-core runs, fixed seeds).
7. Randomness quality of *reshuffle* and *get_random_pair* had to be validated; two Fisher-Yates passes worked better in practice.
8. Repeated evaluation of each algorithm to work properly on each problem instance and with each class and method in the *src/* package.
9. Optimization of *Matrix* class and introduction of statistical metrics.
10. Optimization of delta cost calculations to speedup performance manyfold.
11. Thorough evaluation of possible heuristics to unify definition of hyperparameters of different algorithms for all the problems. For example, derivation of α and number of cooling schedule steps in **SA**, and examination of problems' properties to define elite candidate list lifespan in **TS**.
12. Implementation of **SA** and **TS** in general wasn't very straightforward, and required additional research ([6], [8]).
13. Introducing built-in restarts to local search and random walk algorithms wasn't straightforward and required rigorous prior analysis due to large API functionality.

11 Justification and Future Improvements

Justification. The introduced improvements were necessary to make results valid, comparable, and efficient:

1. Fixed arithmetic issues (overflow) to avoid invalid objective values.
2. Improved reproducibility (fixed seeds, controlled runs) for fair method comparison.
3. Improved random operators quality, which stabilized multi-start behavior.
4. Optimized core implementation to run larger experiments in the same time.
5. Additional caching in cost/delta-cost loops *for* faster iterations was implemented and showed significant speedup in running time of the experiments.
6. Parallel independent restarts *for* lower wall-clock time.

Future Improvements and Expected Effects:

1. Better initialization heuristic (instead of matching single elements rows or columns with the highest/lowest norms can be matched) *for* better starting solutions.
2. Adaptive restart budget instead of fixed restarts *for* better time-quality trade-off.
3. Hybrid methods (e.g., LS + diversification/perturbation) *for* better escape from local minima.

References

- [1] Maciej Komosiński and Maciej Hapke. “Optimization – local search. Lecture notes”. In: (2023). URL: <https://www.cs.put.poznan.pl/mkomosinski/lectures/optimization/LS-en.pdf>.
- [2] “Quadratic assignment problem”. In: *Wikipedia* (). URL: https://en.wikipedia.org/wiki/Quadratic_assignment_problem.
- [3] S.E. KARISCH R.E. BURKARD E. ÇELA and F. RENDL. “QAPLIB – A Quadratic Assignment Problem Library”. In: ().
- [4] *Matplotlib C++*. URL: <https://github.com/lava/matplotlib-cpp>.
- [5] Thomas Stutzle Irina Dumitrescu. “Topics in Local Search”. In: (). URL: <https://www.cs.put.poznan.pl/mkomosinski/lectures/optimization/extras/QAP-fast-update.pdf>.
- [6] Justin Andrew Boyan. *Learning Evaluation Functions for Global Optimization*. School of Computer Science, Carnegie Mellon University, 1998, pp. 187–190.
- [7] Maciej Komosiński. “Lectures on Optimization. Extras. Simulated Annealing with Lam Schedule (pseudocode)”. In: (). URL: <https://www.cs.put.poznan.pl/mkomosinski/lectures/optimization/extras/LamLikeAnnealing.cpp>.
- [8] Manuel Laguna Fred Glover. “Tabu Search: Effective Strategies for Hard Problems in Analytics and Computational Science”. In: *Handbook of Combinatorial Optimization* ().
- [9] Maciej Komosiński and Maciej Hapke. “Optimization – Tabu search. Lecture notes”. In: (). URL: <https://www.cs.put.poznan.pl/mkomosinski/lectures/optimization/TS-en.pdf>.